



SAPIENZA
UNIVERSITÀ DI ROMA

Sapienza University of Rome

Dipartimento di ingegneria informatica, automatica e gestionale "Antonio
Ruberti"

Corso di Master Degree in Artificial Intelligence and Robotics

Multi-agent Decision Support System for Space Mission Operations

Thesis Advisor
Prof. Danilo Comminiello

Candidate
Antonio Lisa Lattanzio
2154208

Academic Year MMXXV-MMXXVI

Abstract

Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Keywords: lorem, ipsum, dolor, sit, amet, consectetur, adipisicing, elit, sed, do.

Contents

List of Figures	iv
List of Tables	v
0.1 The Evolution of Natural Language Processing and Decision Support Systems	1
0.1.1 Early Symbolic Approaches	1
0.1.2 Early Neural Approaches	3
0.1.3 From Static Embeddings to Sequential Modeling	5
0.1.4 The Transformer Revolution	8
0.1.5 Scaling Laws and Emergent Abilities	10
0.1.6 Foundation Models and General-Purpose AI	11
0.1.7 Alignment and Instruction Tuning	11
0.1.8 The Reliability Gap: Hallucinations and Uncertainty	12
0.1.9 Computational Burden and Compression Strategies for LLMs	13
0.1.10 Reasoning Frameworks for Decision Support	14
0.1.11 Long-Context Modeling and Context Management	15
0.1.12 Agentic Large Language Models	16
0.1.13 Evolution of Decision Support Systems	17
Bibliography	19

List of Figures

1	The original Transformer encoder-decoder architecture proposed by Vaswani et al. [47]. The figure illustrates the composition of encoder and decoder stacks, the multi-head attention modules, feed-forward layers, residual connections, and positional encodings. Source: reproduced from [47].	10
2	Unified framework proposed by [48] for LLM-based autonomous agents. Source: Wang et al.[48].	17

List of Tables

1	Bag-of-Words representation of two syntactically different but lexically identical sentences.	1
---	---	---

0.1 The Evolution of Natural Language Processing and Decision Support Systems

The field of Natural Language Processing (NLP) has experienced a remarkable evolution over the past decades. Early approaches relied on manually engineered representations and statistical models, where language was treated as a collection of symbolic features rather than as a structured and contextual system. Advances in machine learning progressively shifted the focus toward distributed representations and neural architectures capable of learning linguistic patterns directly from data. This progression ultimately culminated in the Transformer architecture, which forms the foundation of modern Large Language Models (LLMs). Understanding this historical evolution is essential for appreciating the capabilities and limitations of current AI-based Decision Support Systems (DSS).

The development of NLP can be broadly divided into three major phases. The first phase is characterized by sparse statistical representations such as Bag-of-Words and n-gram language models. The second phase introduced recurrent neural architectures capable of modeling sequential dependencies. Finally, the third phase replaced recurrence with attention mechanisms, enabling scalable contextual reasoning over long sequences.

0.1.1 Early Symbolic Approaches

Early NLP systems were primarily based on symbolic representations and sparse statistical features. Rather than learning semantic representations from data, documents were encoded through manually designed features derived from word occurrences. One of the most influential representations was the *Bag-of-Words* (BoW) model [31, 38]. In this representation, each document is transformed into a vector containing the frequency of every word in a predefined vocabulary, while completely disregarding grammar, syntax, and word order.

For example, consider the following two sentences:

"The doctor treats the patient."

"The patient treats the doctor."

Although the semantic meaning of the two sentences is clearly different, the Bag-of-Words representation assigns them nearly identical vectors because they contain exactly the same set of words. The only retained information is the frequency of each term, whereas all structural information is discarded. Consequently, language is reduced to an unordered multiset of independent lexical features.

This can be explicitly observed in their Bag-of-Words representations, as shown in Table 1.

Term	Sentence 1	Sentence 2
the	2	2
doctor	1	1
patient	1	1
treats	1	1

Table 1: Bag-of-Words representation of two syntactically different but lexically identical sentences.

Although the semantic meaning of the two sentences is clearly different, the Bag-of-Words representation assigns them nearly identical vectors because they contain exactly the same set

of words. The only retained information is the frequency of each term, whereas all structural information is discarded. Consequently, language is reduced to an unordered multiset of independent lexical features.

Despite this limitation, Bag-of-Words proved remarkably effective for several tasks, including document classification, spam detection, topic categorization, and information retrieval. Its simplicity, low computational cost, and compatibility with classical machine learning algorithms such as Naïve Bayes and Support Vector Machines [31] contributed to its widespread adoption.

However, the independence assumption underlying BoW prevents the model from capturing compositional meaning or contextual relationships between words. Expressions such as "artificial intelligence", "credit card", or "New York" are decomposed into independent terms, ignoring the fact that their meaning emerges from the combination of multiple words rather than from each individual token.

The statistical foundations of these early methods were strongly influenced by the *distributional hypothesis*, originally proposed by Harris [19], which states that words occurring in similar linguistic contexts tend to exhibit similar semantic properties. This principle later became one of the cornerstones of statistical NLP and distributional semantics.

Motivated by the need to incorporate sequential information, researchers introduced *n-gram language models* [14]. Instead of treating words as independent observations, n-gram models estimate the probability of a word by conditioning only on the previous $n - 1$ tokens according to the Markov assumption:

$$P(w_t \mid w_1, \dots, w_{t-1}) \approx P(w_t \mid w_{t-(n-1)}, \dots, w_{t-1})$$

Depending on the value of n , different language models can be obtained. A unigram model ($n = 1$) assumes complete independence between words and is therefore equivalent to the Bag-of-Words assumption. A bigram model predicts each word using only the immediately preceding token, whereas a trigram model considers the previous two words.

For instance, given the sentence

"The patient needs medical assistance."

the extracted n-grams are:

- Unigrams: {The, patient, needs, medical, assistance}
- Bigrams: {The patient, patient needs, needs medical, medical assistance}
- Trigrams: {The patient needs, patient needs medical, needs medical assistance}

Unlike Bag-of-Words, these representations preserve a limited notion of local word order, allowing statistical models to learn common expressions, collocations, and short syntactic structures.

Nevertheless, n-gram models remain fundamentally constrained by their fixed context window. Long-range dependencies cannot be modeled because only the previous $n - 1$ tokens are considered. For example, in the sentence

"The physician who examined the patient yesterday **prescribes** the medication."

the relationship between the subject “physician” and the verb “prescribes” spans several intermediate words. A small bigram or trigram model cannot capture such dependencies effectively.

A second limitation concerns scalability. As the value of n increases, the number of possible word combinations grows exponentially. Consequently, even very large corpora contain only a small fraction of all possible n -grams, leading to severe data sparsity problems.

These shortcomings motivated the transition toward neural language models capable of learning continuous distributed representations instead of relying exclusively on explicit frequency counts.

0.1.2 Early Neural Approaches

To overcome the limitations of n -gram models, early neural approaches introduced the idea of learning distributed representations over sequences. Simple Recurrent Networks (SRNs) by Elman [13] established a foundational architecture for processing sequential data and are considered a precursor to modern Recurrent Neural Networks (RNNs).

The training of neural networks, including recurrent architectures, is based on backpropagation [41]. In the recurrent setting, this is extended to Backpropagation Through Time (BPTT) [17], which enables gradient-based optimization over temporal dependencies by unrolling the network across time.

Unlike feedforward models, RNNs process input tokens sequentially while maintaining a hidden state that acts as a memory of past information. A standard formulation of an RNN is given by:

$$\begin{aligned}h_t &= f(W_h h_{t-1} + W_x x_t + b_h) \\ y_t &= W_y h_t + b_y\end{aligned}$$

This formulation allows, in principle, the modeling of arbitrarily long dependencies through the recurrent hidden state. However, training RNNs in practice is challenging due to the vanishing and exploding gradient problems, which limit their ability to preserve information over long sequences.

To understand this issue, consider a simplified recurrent formulation:

$$h_t = \phi(W h_{t-1})$$

Under BPTT, the gradient of the loss with respect to a hidden state can be expressed as a product of Jacobians across time:

$$\frac{\partial L}{\partial h_t} = \frac{\partial L}{\partial h_T} \prod_{k=t}^{T-1} \frac{\partial h_{k+1}}{\partial h_k}$$

Taking the norm yields:

$$\left\| \frac{\partial L}{\partial h_t} \right\| \approx \prod_{k=t}^{T-1} \left\| \frac{\partial h_{k+1}}{\partial h_k} \right\|$$

This shows that the gradient magnitude is determined by repeated multiplication of Jacobian matrices across time steps. Depending on their spectral properties, this product can either grow or decay exponentially, leading respectively to exploding or vanishing gradients.

This fundamental limitation significantly hinders the ability of vanilla RNNs to learn long-term dependencies, and motivates the development of more advanced recurrent architectures designed to improve gradient flow over long sequences.

To address these issues, more advanced architectures were proposed, most notably Long Short-Term Memory (LSTM) networks [21]. LSTMs introduce an explicit memory cell c_t designed to preserve long-term information, together with a set of gating mechanisms that regulate information flow. The core idea is to decouple long-term memory from short-term hidden representations.

The LSTM update equations are defined as:

$$\begin{aligned} I_t &= \sigma(X_t W_{xi} + H_{t-1} W_{hi} + b_i) \\ F_t &= \sigma(X_t W_{xf} + H_{t-1} W_{hf} + b_f) \\ O_t &= \sigma(X_t W_{xo} + H_{t-1} W_{ho} + b_o) \\ \tilde{C}_t &= \tanh(X_t W_{xc} + H_{t-1} W_{hc} + b_c) \\ C_t &= F_t \odot C_{t-1} + I_t \odot \tilde{C}_t \\ H_t &= O_t \odot \tanh(C_t) \end{aligned}$$

The key property of this formulation is the additive update of the cell state c_t , which contrasts with the purely multiplicative transformations of standard RNNs. This structure allows gradients to flow more effectively through time, since the derivative of c_t with respect to c_{t-1} is modulated by the forget gate f_t , which can learn to preserve information when needed.

As a result, LSTMs significantly mitigate the vanishing gradient problem and enable learning of long-range dependencies that are difficult to capture with vanilla RNNs. This makes them particularly effective in sequence modeling tasks such as machine translation, speech recognition, and sequence labeling.

A further simplification of this architecture led to Gated Recurrent Units (GRUs), which combine the forget and input gates into a single update mechanism. GRUs maintain similar performance to LSTMs in many tasks while reducing computational complexity and the number of parameters.

The GRU formulation is defined as:

$$\begin{aligned} Z_t &= \sigma(X_t W_{xz} + H_{t-1} W_{hz} + b_z) \\ R_t &= \sigma(X_t W_{xr} + H_{t-1} W_{hr} + b_r) \\ \tilde{H}_t &= \tanh(X_t W_{xh} + (R_t \odot H_{t-1}) W_{hh} + b_h) \\ H_t &= (1 - Z_t) \odot H_{t-1} + Z_t \odot \tilde{H}_t \end{aligned}$$

Both architectures represent a critical improvement over vanilla RNNs by explicitly addressing the instability of long-term temporal dependencies.

Despite their success, RNN-based architectures still process sequences sequentially, limiting parallelization during training and inference. This inherent inefficiency, combined with residual difficulties in modeling extremely long dependencies, eventually motivated the development of attention-based models and Transformers, which replaced recurrence with global context modeling.

This progression from BoW and n-gram models to RNNs and gated variants marks a fundamental shift in NLP: from static, order-agnostic representations toward dynamic, stateful systems capable of modeling language as a temporal process. This evolution forms the conceptual bridge toward modern contextual embedding methods and Transformer-based architectures.

0.1.3 From Static Embeddings to Sequential Modeling

While recurrent architectures addressed the problem of modeling sequential dependencies, they still relied on input representations that did not explicitly capture semantic similarity between words. This motivated the development of distributed word representations, commonly referred to as word embeddings. Building upon the distributional hypothesis, neural language models sought to learn continuous vector representations in which semantically related words occupy nearby regions of the embedding space. This led to the first fundamental advancement in how machines represent language: the introduction of *Word Embeddings*.

Instead of treating words as isolated discrete entities (as in BoW or classical n-grams), Bengio et al. [3] proposed mapping each word i in a vocabulary V to a continuous, real-valued vector $C(i) \in \mathbb{R}^m$ (where $m \ll |V|$), stored in a shared embedding matrix C . A feedforward neural network then processes the concatenation of these vectors within a fixed context window to predict the probability of the next token w_t via a *softmax* layer. The architecture allows semantically similar words are forced to settle in nearby regions of this continuous vector space, allowing the model to automatically generalize from a training sentence (e.g., "*the cat is walking*") to unseen variations (e.g., "*a dog was running*").

Although theoretically revolutionary, this early model suffered from a massive computational bottleneck caused by computing the softmax normalization over the entire vocabulary $|V|$ at each step. Nevertheless, it laid the conceptual foundations for modern representation learning.

The most influential work in this area is **Word2Vec**, proposed by Mikolov et al. [32]. This model demonstrated that words could be mapped into a high-dimensional vector space where semantic relationships are preserved as geometric distances.

The framework produces these embeddings through two distinct model architectures that leverage local context windows. The Continuous Bag-of-Words (CBOW) model learns representations by predicting a central target word w_t given its surrounding context words within a sliding window of maximum distance C . The vectors of the context words are projected and averaged into a single position, discarding structural word order. The training objective of CBOW is to maximize the average log probability:

$$\mathcal{L}_{\text{CBOW}} = \frac{1}{T} \sum_{t=1}^T \log P(w_t \mid w_{t-C}, \dots, w_{t-1}, w_{t+1}, \dots, w_{t+C}) \quad (1)$$

Conversely, the Continuous Skip-gram model reverses this logic by using the current central word w_t as input to predict the continuous vector representations of its surrounding context words. The Skip-gram objective function maximizes the following conditional log probability:

$$\mathcal{L}_{\text{Skip-gram}} = \frac{1}{T} \sum_{t=1}^T \sum_{-C \leq j \leq C, j \neq 0} \log P(w_{t+j} \mid w_t) \quad (2)$$

The empirical results presented in the study demonstrated significant breakthroughs in both scalability and semantic compositionality. Crucially, the resulting vector space demonstrated linear regularities, meaning that relationships between words were preserved as stable geometric translations. This allowed the models to solve complex semantic and syntactic analogies using simple algebraic operations, such as computing the vector $X = v_{\text{King}} - v_{\text{Man}} + v_{\text{Woman}}$ and finding that its closest cosine distance neighbor was exactly v_{Queen} . Evaluated across extensive benchmark datasets, CBOW achieved state-of-the-art performance on syntactic tasks such as tense and plural inflections, while Skip-gram significantly outperformed all previous architectures on complex semantic questions.

While Word2Vec enabled early NLP tasks to capture basic conceptual analogies, these embeddings remained **static**: a single vector was assigned to a word regardless of its context (e.g., "bank" in a financial vs. geographical sense). Moreover, Word2Vec does not model word order or syntactic structure, since embeddings are learned independently of the surrounding sentence. As a consequence, it cannot capture long-range dependencies or contextual nuances that are essential in many decision-support scenarios. The representation is therefore powerful at the lexical level but fundamentally limited for tasks requiring a deeper understanding of sequence dynamics.

Shortly after, Pennington et al. [36] introduced **GloVe** (Global Vectors for Word Representation), which addresses the structural limitations of local window-based methods by leveraging the global statistical information of the corpus. While Word2Vec relies on local context windows, GloVe explicitly constructs a word-word co-occurrence matrix X , where entries X_{ij} record how frequently word j appears in the context of word i . The core insight of GloVe is that the ratio of co-occurrence probabilities P_{ik}/P_{jk} —rather than the probabilities themselves—carries more meaningful semantic information. To capture this in a linear vector space, the authors propose a weighted least-squares model that seeks to satisfy the following objective:

$$J = \sum_{i,j=1}^V f(X_{ij}) \left(w_i^T \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2 \quad (3)$$

In this formulation, w_i and $\tilde{w}_j$ represent the target and context word vectors, while b_i and $\tilde{b}_j$ are bias terms that account for individual word frequencies. The function $f(X_{ij})$ acts as a weighting mechanism, ensuring that rare co-occurrences are not overweighted, while preventing common word pairs (e.g., "the", "and") from dominating the training objective. By training directly on the non-zero elements of the co-occurrence matrix, GloVe synthesizes the efficiency of global matrix factorization with the superior semantic capture of local window models. Empirical evidence confirmed this hybridization: GloVe demonstrated significant improvements in word analogy tasks and named entity recognition, consistently outperforming its predecessors by producing a vector space with a more robust and meaningful substructure. However, like its predecessors, GloVe remains a **static** embedding model, providing a single vector representation for each word regardless of the polysemy inherent in natural language.

While traditional word embedding models, such as Word2Vec, have proven successful in mapping semantic relationships into continuous vector spaces, they rely on treating words as atomic and isolated entities. This approach ignores the internal morphological structure of language, which represents a significant limitation for morphologically rich languages and results in the challenge

of out-of-vocabulary terms, where rare words fail to acquire reliable representations. To overcome these constraints, the FastText framework [6] was introduced as a powerful extension of the skip-gram architecture that explicitly incorporates subword information. The core innovation of this approach lies in the decomposition of each word into a set of character n-grams. By adding special boundary symbols to distinguish prefixes and suffixes, the model represents every word as the sum of the vector representations of its constituent character n-grams. This design provides several critical advantages that distinguish FastText from previous static embedding methods. First, it ensures high robustness to rare words and previously unseen tokens, as the model can generate meaningful vector representations for any word by composing the vectors of its known n-grams. Furthermore, the model demonstrates a sophisticated morphological awareness, effectively capturing semantic commonalities between inflected forms or complex compound words, an ability that proves particularly advantageous for morphologically rich languages where it consistently outperforms standard word-level approaches.

To address the issue of polysemy and contextual nuance, the field moved toward **Deep Contextualized Word Representations**. A landmark in this transition was **ELMo** (Embeddings from Language Models), proposed by Peters et al. [37]. Unlike static embedding models, ELMo represents each word as a function of its surrounding context by exploiting a deep bidirectional Language Model (biLM). Rather than relying solely on word-level lookup tables, ELMo constructs input representations from character-level convolutions (CNNs), allowing the model to capture rich morphological information and generate meaningful embeddings even for out-of-vocabulary words.

The biLM consists of multiple stacked bidirectional LSTM layers trained with a self-supervised language modeling objective. The forward language model predicts the next token given the preceding context, while the backward language model predicts the previous token given the following context. By processing the sequence in both directions, the model captures both short- and long-range dependencies. Lower layers tend to encode syntactic and morphological information, whereas deeper layers capture increasingly abstract semantic features.

Instead of using only the final hidden layer, ELMo computes the representation of the k -th token as a task-specific weighted combination of all internal biLM representations:

$$\text{ELMo}_k^{\text{task}} = \gamma^{\text{task}} \sum_{j=0}^L s_j^{\text{task}} h_{k,j}^{LM}$$

where $h_{k,j}^{LM}$ denotes the representation of the k -th token produced by the j -th layer of the bidirectional language model, s_j^{task} are task-specific normalized weights learned during downstream training, and γ^{task} is a trainable scaling parameter.

This weighted combination allows downstream models to dynamically exploit both lower-level syntactic information and higher-level semantic knowledge depending on the target task. Consequently, ELMo replaces the idea of a single fixed embedding per word with contextual representations that vary according to the surrounding sentence, substantially improving performance on tasks such as question answering, sentiment analysis, and textual entailment.

Although ELMo represented a major breakthrough by introducing contextual embeddings, it still relied on recurrent architectures to model sequences. Consequently, contextual representations had to be computed sequentially, limiting parallelization and making long-range dependency modeling computationally expensive. These limitations ultimately motivated the development of the

Transformer architecture, which replaced recurrence with self-attention.

0.1.4 The Transformer Revolution

The definitive breakthrough occurred with the publication of "*Attention Is All You Need*" [47]. Vaswani et al. proposed a novel architecture entirely based on the **Self-Attention mechanism**, dispensing with recurrence and convolutions. The core innovation lies in the model's ability to weigh the significance of different parts of the input data independently of their distance in the sequence. Mathematically, the Scaled Dot-Product Attention is defined as:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (4)$$

where Q , K , and V represent the Query, Key, and Value matrices, and d_k is the dimension of the keys. The scaling factor $\sqrt{d_k}$ plays a crucial role in stabilizing the optimization process. Without this normalization term, the dot products between high-dimensional query and key vectors could grow excessively large, causing the softmax function to produce extremely peaked probability distributions and resulting in vanishing gradients during training. The scaling operation therefore contributes to more stable learning dynamics and improved convergence properties.

Conceptually, the attention mechanism can be interpreted as a weighted aggregation process. Each query vector determines how much importance should be assigned to all available key-value pairs, enabling the model to selectively focus on the most relevant contextual information for a given token. This selective processing capability constitutes one of the fundamental reasons behind the remarkable effectiveness of Transformer-based architectures.

This mechanism allows the model to build a global understanding of the input, enabling the emergence of "contextual intelligence" where word representations change dynamically based on the surrounding text. In the context of NLP, this marked the transition from local pattern matching toward richer contextual representations capable of modeling long-range semantic and syntactic dependencies. These representations later proved essential for the emergence of increasingly sophisticated reasoning-like behaviors in large-scale language models. A key extension introduced in the original Transformer architecture is the multi-head self-attention mechanism. Instead of computing a single attention map, the model projects the input into multiple sets of query, key, and value spaces and applies the attention function in parallel across these sets.

Formally, multi-head attention is defined as:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O \quad (5)$$

where

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \quad (6)$$

and W_i^Q , W_i^K , W_i^V , and W^O are learnable projection matrices. This formulation enables each attention head to specialize in capturing different forms of dependencies within the data, thereby enriching the overall representation learned by the model.

The outputs from these attention heads are then concatenated and linearly transformed, allowing the model to jointly attend to diverse types of relationships in the input sequence, such as syntactic

and semantic patterns simultaneously. By combining multiple attention heads, the network can capture richer representations than a single attention head would allow, without increasing the overall model depth.

Each Transformer block combines multi-head self-attention with a position-wise feed-forward network (FFN). Residual connections and layer normalization are applied around each sub-layer, facilitating stable optimization and enabling the successful training of very deep architectures.

Although self-attention enables every token to attend to all other tokens in the sequence, the mechanism itself is inherently permutation invariant and therefore provides no information about token order. To preserve the sequential nature of language, the original Transformer introduces **Positional Encodings**, which are added to the input embeddings before they are processed by the encoder or decoder.

Rather than learning positional representations, Vaswani et al. proposed deterministic sinusoidal functions capable of encoding both absolute and relative positions within a sequence:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right) \quad (7)$$

$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right) \quad (8)$$

where pos denotes the token position and d_{model} is the embedding dimension. This formulation allows the model to infer relative distances between tokens while also enabling generalization to sequence lengths not encountered during training. Although modern large language models often replace sinusoidal encodings with learned positional embeddings or rotary positional embeddings (RoPE), the original formulation remains a foundational contribution that demonstrated how sequential information could be incorporated without relying on recurrent architectures.

Beyond the attention mechanism itself, the original Transformer introduced a modular encoder-decoder architecture, illustrated in Figure 1. The encoder consists of a stack of identical layers, each composed of a multi-head self-attention module followed by a position-wise feed-forward network. Residual connections and layer normalization surround each sub-layer, improving optimization stability and enabling deeper architectures.

The decoder follows a similar structure but incorporates two important differences. First, it employs masked self-attention to prevent each output token from attending to future positions during training. Second, an additional cross-attention layer allows the decoder to attend to the contextual representations generated by the encoder, enabling effective sequence-to-sequence learning for tasks such as machine translation.

This modular design later gave rise to the dominant language model families. Encoder-only architectures, such as BERT, specialize in language understanding tasks, while decoder-only architectures, such as the GPT series, are optimized for autoregressive text generation. Encoder-decoder models, including T5, remain particularly effective for sequence transformation tasks such as translation and summarization.

Another important practical advantage of Transformer architectures is their massive parallelization potential. Unlike recurrent networks, which process tokens sequentially, each attention layer in a Transformer can compute interactions between all positions in the input sequence simultaneously. As a result, Transformers can be trained more efficiently on large corpora, enabling the scale-up

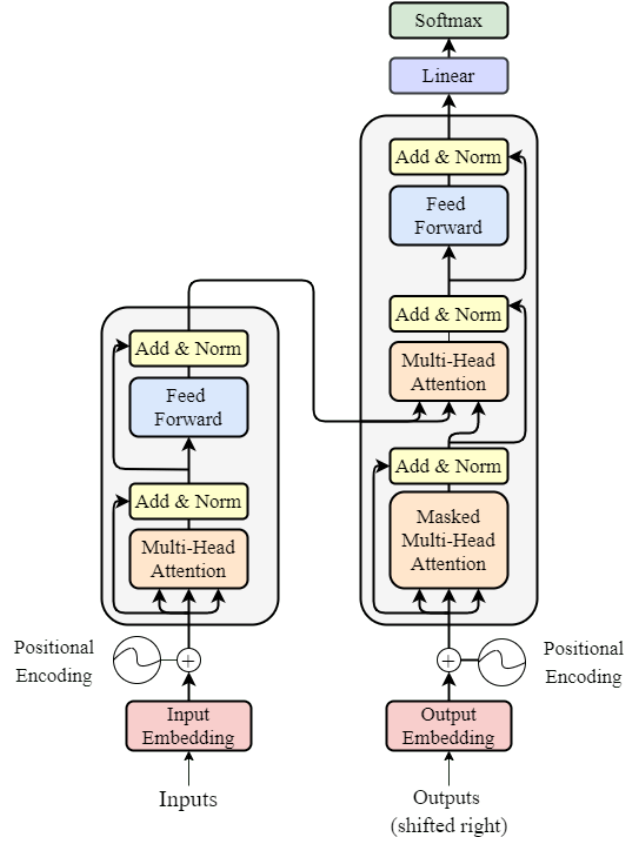


Figure 1: The original Transformer encoder-decoder architecture proposed by Vaswani et al. [47]. The figure illustrates the composition of encoder and decoder stacks, the multi-head attention modules, feed-forward layers, residual connections, and positional encodings. Source: reproduced from [47].

that underpins modern large language models.

Despite its advantages, the standard self-attention mechanism exhibits quadratic time and memory complexity with respect to the input sequence length, as every token attends to every other token. This limitation has motivated extensive research into efficient attention mechanisms like flash attention [11], which uses tiling to reduce the number of memory reads/writes, or linformer [49], that approximate the standard attention mechanism with low rank matrices.

Collectively, these innovations established the Transformer as the dominant neural architecture for NLP and, subsequently, for a wide range of machine learning applications. More importantly, its scalability and training efficiency made it possible to train models on unprecedented amounts of data and parameters, laying the foundation for the emergence of modern Large Language Models.

0.1.5 Scaling Laws and Emergent Abilities

The evolution culminated in the transition from task-specific models to *Large Language Models* (LLMs). This shift was driven by the discovery of scaling laws [26], which suggested that model performance, and consequently capabilities on NLP tasks, improve with increases in parameter count, dataset size, and compute, which can indirectly improve the performance of systems that leverage NLP components, including DSS applications.

Furthermore, the introduction of the Pre-training and Fine-tuning paradigm, popularized by BERT [12] and the GPT series [39], enabled these systems to internalize vast amounts of world

knowledge. This trajectory led to the discovery of *emergent abilities*: capabilities not explicitly present in smaller models, such as in-context learning and multi-step logical reasoning [51]. These behaviors are referred to as emergent because they are not explicitly programmed nor consistently observed in smaller models, but rather appear once the model exceeds certain scales of parameters and training data. Although the precise mechanisms underlying emergence remain an active area of research, empirical evidence suggests that increasing scale enables qualitatively new capabilities beyond simple improvements in benchmark accuracy. For modern Decision Support Systems, this represents a transition from "engines that process text" to systems that can generate knowledge-informed recommendations, while human oversight remains necessary.

0.1.6 Foundation Models and General-Purpose AI

The scalability of the Transformer architecture, coupled with the empirical discovery of scaling laws, has catalyzed a paradigm shift in artificial intelligence: the emergence of **Foundation Models**. As defined by Bommasani et al. [7], a foundation model is any model trained on a vast amount of data, typically using self-supervision at scale, that can be adapted (e.g., fine-tuned) to a wide range of downstream tasks. This represents a fundamental departure from the traditional AI development lifecycle, where models were painstakingly engineered for single, narrow objectives. Instead, foundation models serve as a versatile base, encoding broad representations of language, logic, and world knowledge that can be repurposed across diverse domains, including the sophisticated requirements of Decision Support Systems.

A pivotal demonstration of this potential was provided by Brown et al. [8] with the introduction of GPT-3. This work highlighted that when language models are scaled to billions of parameters, they begin to exhibit *in-context learning* capabilities. Unlike previous approaches that required gradient-based fine-tuning to perform a new task, GPT-3 demonstrated the ability to adapt to specific instructions simply by observing a few examples provided within the input prompt (few-shot learning). This shift fundamentally changed the interaction between users and systems, moving from "training" to "prompting" as the primary mechanism for directing model behavior.

However, the rapid expansion of model size brought to light significant challenges regarding computational efficiency. The industry trend initially focused on increasing parameter counts indiscriminately; this was challenged by Hoffmann et al. [22] in their study on *Chinchilla*. The authors demonstrated that the performance of a language model is not solely dependent on its parameter count, but rather on an optimal balance between model size and the volume of training data. Their analysis showed that, for a fixed computational budget, allocating additional resources to training data rather than solely increasing parameter count yielded substantially better performance. This finding established new scaling laws, suggesting that most contemporary models were significantly under-trained. This insight provided a roadmap for developing "compute-optimal" models, enabling superior performance with more efficient resource allocation.

0.1.7 Alignment and Instruction Tuning

Although large language models acquired broad linguistic knowledge through large-scale pre-training, they were not inherently optimized to follow user instructions or assist with practical tasks like Decision Support Systems, this powerful capability was enabled by the development of *Alignment*

techniques. While base LLMs possess extensive statistical information derived from their training corpora, they often fail to follow specific user intents or produce helpful, safe, and concise reasoning. To bridge this gap, **Instruction Tuning** was introduced, most notably through the work on **InstructGPT** by Ouyang et al. [34]. This process involves **fine-tuning** the model on a dataset of human-written prompt-response pairs, instruction tuning changes the training distribution by exposing the model to datasets composed of instructions paired with high-quality responses, thereby teaching it to better interpret and follow user requests.

A critical component of this alignment is **Reinforcement Learning from Human Feedback (RLHF)**. This methodology, popularized by Christiano et al. [10] and later refined for LLMs [45], allows models to optimize their outputs based on human preferences. The RLHF pipeline typically consists of three stages. First, a supervised fine-tuning phase is performed, where the initial language model is trained on high-quality demonstrations generated by human annotators. Second, a reward model is trained from human preference comparisons, where evaluators rank alternative model outputs according to criteria such as helpfulness, relevance, and overall quality. Finally, the language model is optimized against the learned reward model using reinforcement learning, typically through Proximal Policy Optimization (PPO) [43]. By incorporating human preferences into the optimization process, RLHF guides the model toward outputs that better satisfy alignment objectives, although limitations such as hallucinations, biases, and sensitivity to reward model imperfections remain. While RLHF has been the industry standard for alignment, its reliance on a separately trained reward model and the computational complexity of the Proximal Policy Optimization (PPO) algorithm have led to the development of more stable alternatives. A prominent breakthrough is **Direct Preference Optimization (DPO)**, introduced by Rafailov et al. [40]. DPO simplifies the alignment process by mathematically deriving a direct mapping between the reward function and the optimal policy, effectively bypassing the need for a separate reward model and reinforcement learning training loops. By optimizing the language model directly on preference data via a simple classification loss, DPO offers a computationally efficient and more stable approach to alignment, which has rapidly become a preferred method for fine-tuning modern Large Language Models.

0.1.8 The Reliability Gap: Hallucinations and Uncertainty

One of the primary obstacle to the integration of LLMs in critical Decision Support Systems (DSS) is the phenomenon of **hallucinations**, defined as the generation of factually incorrect or ungrounded statements despite high linguistic fluency [24]. This stems from the underlying training objective of next-token log-likelihood, which prioritizes plausible linguistic continuation over factual verification. In a DSS context, **extrinsic hallucinations**, where the output cannot be verified from provided sources or real-world knowledge, are particularly hazardous, as the model may provide a confident but false justification for a recommendation [2].

Beyond explicit errors, a critical risk is the **lack of self-awareness regarding uncertainty**. State-of-the-art models are often poorly calibrated, meaning their internal probability estimates do not align with their actual correctness [25].

Recent surveys [1] identify a broad taxonomy of **hallucination detection methods**, designed to assess the factuality and faithfulness of LLM outputs before they are used in high-stakes Decision Support Systems.

- **Retrieval-based methods** detect hallucinations by comparing generated statements against external knowledge sources or retrieved evidence. These techniques, often implemented through RAG pipelines [27], are particularly effective for identifying *extrinsic hallucinations* by checking whether claims are supported by authoritative documents.
- **Uncertainty-based approaches** estimate unreliability through internal model signals such as token-level probabilities, entropy, attention stability, or structured uncertainty propagation. This line of work directly addresses the calibration issues highlighted in [25], offering a principled way to detect when a model is uncertain despite fluent output.
- **Embedding-based methods** measure semantic alignment between inputs, outputs, and reference sources within a shared embedding space. Low semantic similarity indicates drift from the provided context or task grounding.
- **Learning-based detectors**, including supervised, weakly supervised, and agent-based systems, train dedicated models to classify hallucinated versus factual outputs. These techniques capture complex error patterns that cannot be identified through simple probability thresholds or similarity metrics.
- **Self-consistency-based techniques** generate multiple answers or paraphrased queries to assess stability across model outputs. Inconsistencies in meaning or reasoning serve as indicators of potential hallucinations, making these methods valuable when external verification is unavailable.

Overall, this taxonomy underscores that mitigating hallucinations involves not only filtering incorrect content but also understanding where and why LLM reasoning becomes unreliable, which is essential for safe integration into critical DSS.

Recent work suggests that some failure modes traditionally framed as hallucinations may come not only from generative uncertainty but also from structural challenges in long-context evidence aggregation [5]. Studies on retrieval in extended contexts show that the size and distribution of relevant information influence an LLM’s ability to locate and integrate the correct evidence. When small but crucial “gold” contexts are embedded within much longer distractor passages, models exhibit reduced aggregation accuracy and increased positional sensitivity. These effects are particularly relevant for retrieval-augmented and agentic systems, where evidence sources often vary considerably in length. In such settings, short but important documents may be underutilized, potentially leading the model to produce unsupported outputs even when the necessary information is included in the input. These findings suggest that hallucination-related risks arise not only from calibration issues or insufficient grounding but also from context-size asymmetry and the design of evidence-aggregation pipelines.

0.1.9 Computational Burden and Compression Strategies for LLMs

Even though LLMs are extremely useful they also are computationally expensive, the development of new models has been characterized by an increase of the parameters size that grown to hundreds of billions of parameters in the last years, with hardware that are getting more and more powerful

and optimized to keep up with the evolution of this technology [28]. The main methods to reduce the cost are applicable during the fine-tuning stage or the inference task:

- **Parameter-efficient fine-tuning (PEFT)**: refers to the process of adjusting the parameters of a pre-trained large model to adapt it to a specific task or domain while minimizing the number of additional parameters introduced or computational resources required [18]. One of their main methods is LoRA [23] which freezes the pre-trained model weights and injects trainable rank decomposition matrices into each layer of the Transformer architecture, reducing the number of trainable parameters.
- **Parameters quantization**: focuses on reducing the size of the parameters to usually 4 or 8 bits like Smoothquant [53], representative post-training quantization methods include [15], which quantizes weights block-wise and iteratively corrects quantization errors using Hessian information, and Activation-aware Weight Quantization (AWQ), which focuses on reducing the error by preserving the most important weights [29].
- **Quantization Aware Fine Tuning**: QA-Lora represents a hybrid approach between PEFT and quantization, where the model is fine-tuned while accounting for quantization constraints [54].
- **Pruning**: focus on lightening the model size by removing the connections or neurons that are less important [16, 44]
- **Knowledge Distillation**: is a technique where a smaller model (the student) is trained to replicate the behavior of a larger model (the teacher), effectively transferring knowledge from the teacher to the student [20, 42].

Compression techniques inevitably involve trade-offs between efficiency and performance, as aggressive quantization, pruning, or parameter-efficient fine-tuning may degrade model accuracy or require additional implementation effort, and task-specific performance variability must be considered [15, 16, 23, 29].

0.1.10 Reasoning Frameworks for Decision Support

Beyond architectural and alignment improvements, the operational use of LLMs in decision-making has been revolutionized by specialized prompting and reasoning frameworks. The introduction of **Chain-of-Thought (CoT)** prompting [52] demonstrated that forcing a model to generate intermediate reasoning steps significantly enhances performance on complex logical tasks.

While CoT provides a "slow thinking" approach essential for auditing a DSS, relying on a single greedy-decoded trajectory can lead to error propagation. To mitigate this, **Self-Consistency** [50] introduces a decoding strategy that samples multiple reasoning paths and selects the most frequent marginal result, leveraging the intuition that complex problems often admit multiple paths to a unique correct answer.

Further advancing non-linear reasoning, the **Tree of Thoughts (ToT)** framework [55] generalizes the CoT paradigm by representing intermediate reasoning steps as discrete "thoughts" organized in a tree structure. Unlike CoT, where the model commits to a single sequential chain of deductions, ToT enables the exploration of multiple candidate reasoning trajectories through a search process. At each step, the LLM can generate alternative intermediate states, evaluate their quality through

self-assessment or external feedback mechanisms, and prune unpromising branches. This approach introduces a form of deliberative reasoning closer to classical search algorithms, allowing the model to backtrack, compare alternative hypotheses, and allocate additional computational effort to particularly challenging decisions. In the context of Decision Support Systems (DSS), ToT is especially valuable for scenarios characterized by uncertainty, long planning horizons, or conflicting objectives, where premature commitment to a single reasoning path may lead to suboptimal recommendations.

Building upon this idea, the **Graph of Thoughts (GoT)** framework [4] relaxes the hierarchical constraints of tree-based exploration by modeling reasoning processes as directed graphs rather than strictly branching structures. While ToT assumes that each new thought derives from a single previous state, GoT allows arbitrary dependencies between intermediate reasoning units, enabling operations such as aggregation, merging, refinement, and transformation of partial solutions. This graph-based representation better captures complex problem-solving processes in which multiple reasoning paths can interact and contribute jointly to a final decision. For DSS applications, GoT provides a more flexible framework for representing interconnected evidence, alternative hypotheses, and multi-stage decision processes, where information may need to be combined, revisited, or propagated across different reasoning components.

The **ReAct** framework [56] represents a fundamental transition from isolated reasoning toward agentic decision-making by interleaving reasoning and action. Instead of generating a final answer based solely on the information encoded in the model parameters, ReAct structures the interaction as an iterative loop composed of three main phases: reasoning, action, and observation. The model first generates a reasoning trace to determine the next objective or strategy, then executes an external action, such as querying a database, retrieving documents, invoking an API, or interacting with a software environment, and finally incorporates the resulting observations into subsequent reasoning steps.

This mechanism allows LLMs to dynamically acquire information, verify intermediate assumptions, and adapt their behavior according to external feedback. Compared with purely reasoning-oriented approaches such as CoT or ToT, ReAct introduces an explicit connection between cognitive processes and environmental interaction, reducing the limitations caused by static knowledge and closed-world assumptions. For Decision Support Systems, this capability is particularly relevant because decisions often depend on continuously changing data sources, domain-specific tools, and feedback from operational environments.

When integrated with Retrieval-Augmented Generation (RAG) [27], ReAct-based systems can combine structured reasoning with external knowledge retrieval, enabling LLMs to identify information gaps, retrieve relevant evidence, evaluate available alternatives, and progressively refine recommendations. This combination shifts LLMs from static predictive models toward interactive reasoning agents capable of supporting complex decision processes in dynamic and uncertain environments.

0.1.11 Long-Context Modeling and Context Management

While early Transformer-based models were limited to relatively short context windows, modern LLMs have progressively expanded their capacity to process longer sequences. This evolution enables applications requiring the analysis of extensive documents, multi-turn conversations, and complex workflows involving heterogeneous information sources. However, the standard Transformer archi-

texture suffers from a quadratic computational and memory complexity, $O(n^2)$, relative to sequence length n . This constraint turns the context window into a significant bottleneck: as sequence length increases, the memory footprint of the Key-Value (KV) cache grows linearly, often exhausting GPU VRAM during inference, while the self-attention mechanism becomes the primary computational latency driver. To overcome these limitations, recent research has focused on architecture and positional encoding advancements.

Initial approaches relied on linear attention approximations, but many state-of-the-art models primarily adopt **Rotary Positional Embeddings (RoPE)** [46] combined with scaling techniques. Works such as Position Interpolation [9] and **YaRN** [35] have demonstrated that context windows can be extended far beyond pre-training limits by mathematically re-scaling the rotational frequency of embeddings, allowing models to 'perceive' longer sequences without extensive retraining.

Another problem is the increased risk of hallucinations and reasoning errors when the model is forced to process long contexts, as the relevant information may be diluted among irrelevant content. This is particularly problematic in Decision Support Systems, where critical evidence may be embedded within lengthy documents or multi-source data streams. The work of Liu et al. [30] demonstrates that current language models, including those explicitly designed to handle long contexts, struggle to robustly retrieve and use information located in the middle of their input window. Instead of treating all parts of the context equally, these models exhibit a U-shaped performance curve where performance is highest when relevant information is placed at the very beginning (primacy bias) or at the very end (recency bias) of the input. Performance drops significantly when the model is forced to access information located in the middle of the context. In some cases, performance is even worse than if the model had been given no context at all.

0.1.12 Agentic Large Language Models

The evolution from Large Language Models to Agentic Large Language Models represents a shift from passive text generation toward autonomous task execution. While traditional LLMs operate primarily as reactive systems that map user prompts to generated responses, agentic architectures embed language models within broader computational systems that provide planning, memory, action executions, and profiling modules [48].

The **profiling module** defines the role, objectives, capabilities, and behavioral constraints of the agent. By specifying characteristics such as domain expertise, task objectives, or interaction style, the profile conditions the reasoning process and influences planning, memory retrieval, and action selection.

The **memory module** can store data in several formats, each offering unique trade-offs between semantic richness, retrieval efficiency, and ease of manipulation: Natural Language, Embeddings, Databases, Structured Lists, or hybrid. The selection of data is typically governed by three criteria: recency, relevance, and importance.

The **planning module** is responsible for managing long-horizon tasks, orchestrating complex workflows, and ensuring coherent action sequences across multiple steps. The planning can be performed without feedback, where the agent does not get a feedback that influences its behaviour or with feedback for tasks where initial plans frequently fail due to unforeseen environmental dynamics or overly complex preconditions. The feedback can be provided by the environment, by other agents, or by a human-in-the-loop.

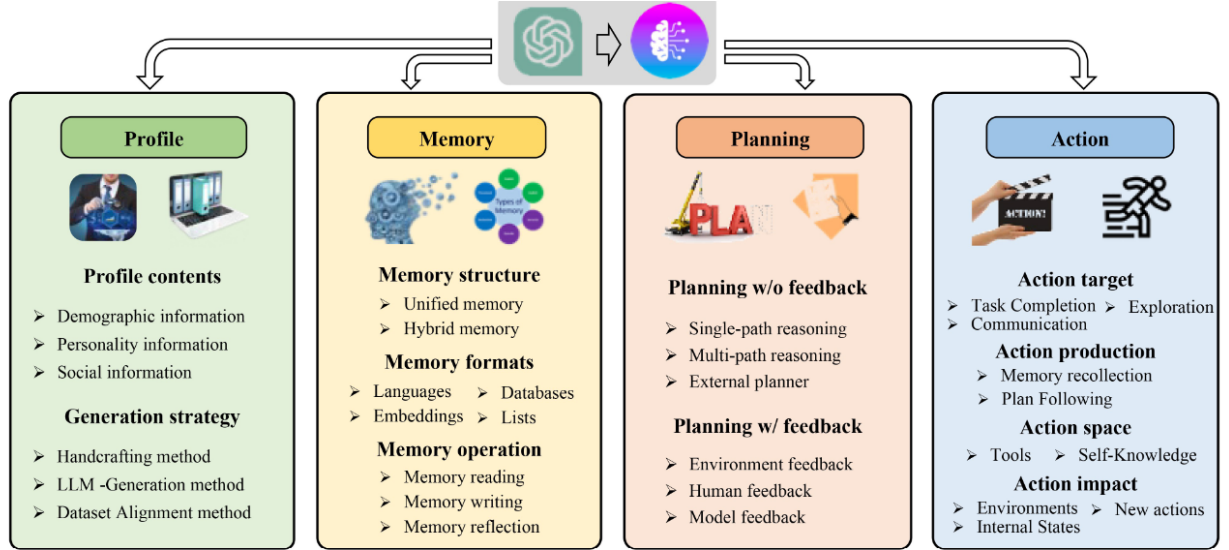


Figure 2: Unified framework proposed by [48] for LLM-based autonomous agents. Source: Wang et al.[48].

The action module serves as the final downstream component of an autonomous agent architecture, translating high-level decisions into tangible outcomes through direct environmental interaction.

An agent's ability to execute these actions relies on a synergy between the LLM's inherent reasoning and a suite of external resources:

- **Tool Integration:** External tools bridge critical gaps in an LLM's capabilities, such as mitigating hallucinations or providing domain-specific expertise.
- **APIs:** Agents utilize APIs to interface with web services, calculators, compilers, and specialized software libraries, enabling them to perform precise, executable tasks.
- **Databases and Knowledge Bases:** allow agents to query structured data, ensuring that actions are grounded in verifiable, external information.
- **External Models:** Agents can offload complex or multimodal processing to specialized supplementary models.

0.1.13 Evolution of Decision Support Systems

The state-of-the-art in this domain has recently moved toward *adaptive, multi-agent architectures* that operationalize these reasoning frameworks within high-stakes environments. A primary example is the work of Ögdü et al. [33], who developed a multi-layered, adaptive Clinical Decision Support System (CDSS). Unlike traditional static models, this architecture integrates biomedical RAG with real-time web intelligence through a collaborative framework of autonomous agents. The system introduces several key innovations:

- **Timeliness and Evidence Freshness:** By incorporating a dedicated web-intelligence agent, the system mitigates the "knowledge cutoff" problem of static LLMs, ensuring recommendations are grounded in the latest clinical guidelines.

- **Confidence-Aware Optimization:** The system utilizes an adaptive feedback loop that triggers re-querying and hypothesis refinement whenever internal confidence scores fall below a defined threshold ($\tau = 0.65$), effectively automating the iterative reasoning seen in ToT/GoT frameworks.
- **Domain-Specific Grounding:** To ensure precision in medical contexts, the system employs a specialized BioBERT-based vector database for processing complex clinical jargon.

Validated on the MedQA and PubMedQA datasets—achieving accuracies of 94% and 88% respectively, this approach demonstrates how the integration of multi-source data fusion and adaptive optimization loops can overcome the reliability barriers of standard LLM-based reasoning.

Bibliography

- [1] A. Alansari and H. Luqman. Large language models hallucination: A comprehensive survey. *arXiv preprint arXiv:2510.06265*, 2025.
- [2] D. Anh-Hoang, V. Tran, and L.-M. Nguyen. Survey and analysis of hallucinations in large language models: attribution to prompting strategies or model behavior. *Frontiers in Artificial Intelligence*, Volume 8 - 2025, 2025.
- [3] Y. Bengio, R. Ducharme, P. Vincent, and C. Jauvin. A neural probabilistic language model. *Journal of machine learning research*, 3(Feb):1137–1155, 2003.
- [4] M. Besta, N. Blach, A. Kubicek, R. Gerstenberger, M. Podstawski, L. Gianinazzi, J. Gajda, T. Lehmann, H. Niewiadomski, P. Nyczyk, et al. Graph of thoughts: Solving elaborate problems with large language models. In *Proceedings of the AAAI conference on artificial intelligence*, volume 38, pages 17682–17690, 2024.
- [5] O. Bianchi, M. J. Koretsky, M. Willey, C. X. Alvarado, T. Nayak, A. Asija, N. Kuznetsov, M. A. Nalls, F. Faghri, and D. Khashabi. Hidden in the haystack: Smaller needles are more difficult for llms to find. *arXiv preprint arXiv:2505.18148*, 2025.
- [6] P. Bojanowski, E. Grave, A. Joulin, and T. Mikolov. Enriching word vectors with subword information. *CoRR*, abs/1607.04606, 2016.
- [7] R. Bommasani, D. A. Hudson, E. Adeli, R. Altman, S. Arora, S. von Arx, M. S. Bernstein, J. Bohg, A. Bosselut, E. Brunskill, et al. On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258*, 2021.
- [8] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- [9] S. Chen, S. Wong, L. Chen, and Y. Tian. Extending context window of large language models via positional interpolation, 2023.
- [10] P. F. Christiano, J. Leike, T. Brown, M. Martic, S. Legg, and D. Amodei. Deep reinforcement learning from human preferences. *Advances in neural information processing systems*, 30, 2017.
- [11] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 16344–16359. Curran Associates, Inc., 2022.

- [12] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pages 4171–4186, 2019.
- [13] J. L. Elman. Finding structure in time. *Cognitive science*, 14(2):179–211, 1990.
- [14] G. A. Fink. n-gram models. In *Markov Models for Pattern Recognition: From Theory to Applications*, pages 107–127. Springer, 2014.
- [15] E. Frantar, S. Ashkboos, T. Hoefer, and D. Alistarh. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2022.
- [16] M. Gordon, K. Duh, and N. Andrews. Compressing bert: Studying the effects of weight pruning on transfer learning. In *Proceedings of the 5th Workshop on Representation Learning for NLP*, pages 143–155, 2020.
- [17] R. Grosse. Lecture 15: Exploding and vanishing gradients. *University of Toronto Computer Science*, 2017.
- [18] Z. Han, C. Gao, J. Liu, J. Zhang, and S. Q. Zhang. Parameter-efficient fine-tuning for large models: A comprehensive survey. *arXiv preprint arXiv:2403.14608*, 2024.
- [19] Z. S. Harris. Distributional structure. *Word*, 10(2-3):146–162, 1954.
- [20] G. Hinton, O. Vinyals, and J. Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.
- [21] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.
- [22] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. Casas, L. A. Hendricks, J. Welbl, A. Clark, et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 10, 2022.
- [23] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen, et al. Lora: Low-rank adaptation of large language models. *Iclr*, 1(2):3, 2022.
- [24] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung. Survey of hallucination in natural language generation. *ACM computing surveys*, 55(12):1–38, 2023.
- [25] Z. Jiang, J. Araki, H. Ding, and G. Neubig. How can we know when language models know? on the calibration of language models for question answering. *Transactions of the Association for Computational Linguistics*, 9:962–977, 09 2021.
- [26] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

- [27] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.
- [28] J. Li, J. Xu, S. Huang, Y. Chen, W. Li, J. Liu, Y. Lian, J. Pan, L. Ding, H. Zhou, et al. Large language model inference acceleration: A comprehensive hardware perspective. *arXiv preprint arXiv:2410.04466*, 2024.
- [29] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han. Awq: Activation-aware weight quantization for on-device llm compression and acceleration. In P. Gibbons, G. Pekhimenko, and C. D. Sa, editors, *Proceedings of Machine Learning and Systems*, volume 6, pages 87–100, 2024.
- [30] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.
- [31] C. D. Manning. *Introduction to information retrieval*. Syngress Publishing,, 2008.
- [32] T. Mikolov, K. Chen, G. Corrado, and J. Dean. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*, 2013.
- [33] Ç. U. Öğdü, K. Arslanoğlu, and M. Karaköse. An adaptive multi-agent llm-based clinical decision support system integrating biomedical rag and web intelligence. *IEEE Access*, 2025.
- [34] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.
- [35] B. Peng, J. Quesnelle, H. Fan, and E. Shippole. Yarn: Efficient context window extension of large language models, 2023.
- [36] J. Pennington, R. Socher, and C. D. Manning. Glove: Global vectors for word representation. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, pages 1532–1543, 2014.
- [37] M. E. Peters, M. Neumann, M. Iyyer, M. Gardner, C. Clark, K. Lee, and L. Zettlemoyer. Deep contextualized word representations. In M. Walker, H. Ji, and A. Stent, editors, *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, pages 2227–2237, New Orleans, Louisiana, June 2018. Association for Computational Linguistics.
- [38] W. A. Qader, M. M. Ameen, and B. I. Ahmed. An overview of bag of words; importance, implementation, applications, and challenges. In *2019 international engineering conference (IEC)*, pages 200–204. IEEE, 2019.
- [39] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.

-
- [40] R. Rafailov, A. Sharma, E. Mitchell, C. D. Manning, S. Ermon, and C. Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in neural information processing systems*, 36:53728–53741, 2023.
 - [41] D. E. Rumelhart, G. E. Hinton, and R. J. Williams. Learning representations by back-propagating errors. *nature*, 323(6088):533–536, 1986.
 - [42] V. Sanh, L. Debut, J. Chaumond, and T. Wolf. Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019.
 - [43] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
 - [44] S. T. Sreenivas, S. Muralidharan, R. Joshi, M. Chochowski, A. S. Mahabaleshwarkar, G. Shen, J. Zeng, Z. Chen, Y. Suhara, S. Diao, et al. Llm pruning and distillation in practice: The minitron approach. *arXiv preprint arXiv:2408.11796*, 2024.
 - [45] N. Stiennon, L. Ouyang, J. Wu, D. Ziegler, R. Lowe, C. Voss, A. Radford, D. Amodei, and P. F. Christiano. Learning to summarize with human feedback. *Advances in neural information processing systems*, 33:3008–3021, 2020.
 - [46] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.
 - [47] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
 - [48] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345, 2024.
 - [49] S. Wang, B. Z. Li, M. Khabsa, H. Fang, and H. Ma. Linformer: Self-attention with linear complexity. *arXiv preprint arXiv:2006.04768*, 2020.
 - [50] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2022.
 - [51] J. Wei, Y. Tay, R. Bommasani, C. Raffel, B. Zoph, S. Borgeaud, D. Yogatama, M. Bosma, D. Zhou, D. Metzler, et al. Emergent abilities of large language models. *arXiv preprint arXiv:2206.07682*, 2022.
 - [52] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.
 - [53] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han. Smoothquant: Accurate and efficient post-training quantization for large language models. In *International conference on machine learning*, pages 38087–38099. PMLR, 2023.
-

- [54] Y. Xu, L. Xie, X. Gu, X. Chen, H. Chang, H. Zhang, Z. Chen, X. Zhang, and Q. Tian. Qa-lora: Quantization-aware low-rank adaptation of large language models. *arXiv preprint arXiv:2309.14717*, 2023.
- [55] S. Yao, D. Yu, J. Zhao, I. Shafran, T. Griffiths, Y. Cao, and K. Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems*, 36:11809–11822, 2023.
- [56] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. R. Narasimhan, and Y. Cao. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*, 2022.